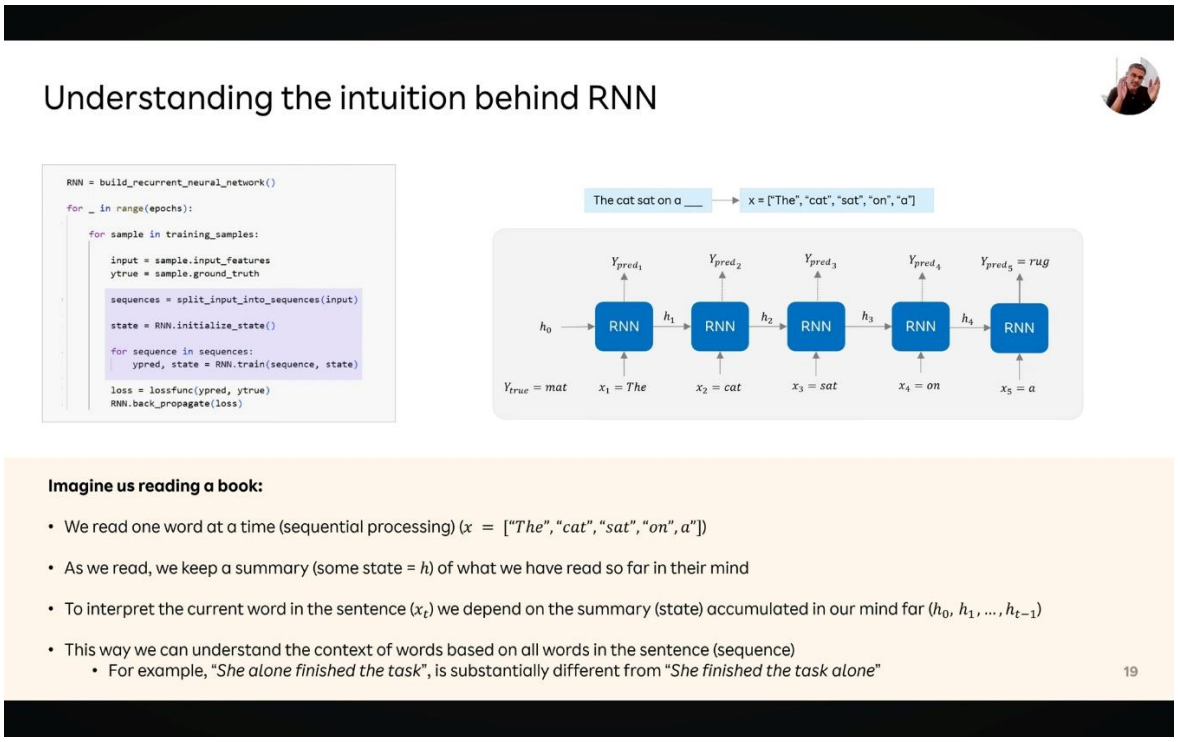
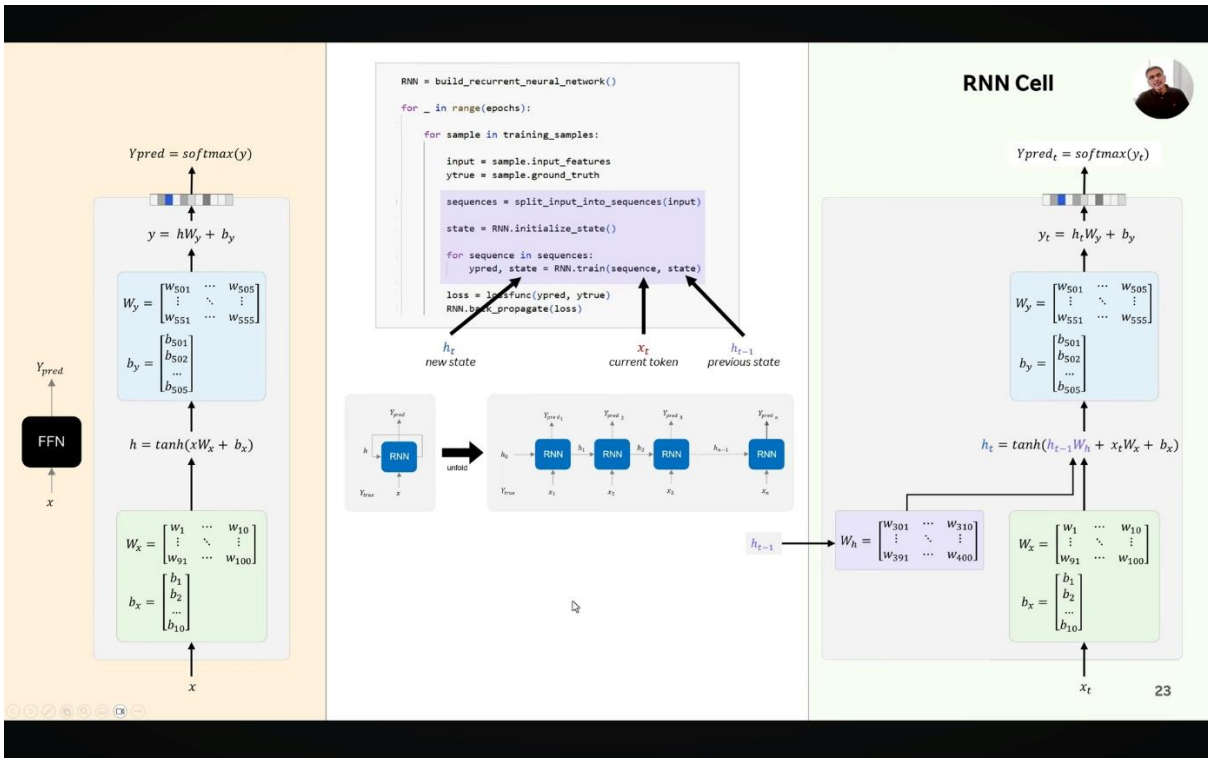
RNN

A Recurrent Neural Network (RNN) is a class of neural networks designed for modelling sequential data by maintaining a hidden state that captures information from previous time steps, enabling the network to learn temporal dependencies through recursive parameter sharing across sequence positions.

WORKING STEPS ARE -





⚡ One-Line Intuition

👉 *"At each time step, RNN updates its memory using current input and past memory, then produces an output."*

CHARACTERISTICS OF RNN

1.Sequential Processing

RNN processes inputs one token at a time in order ($x_1 \rightarrow x_2 \rightarrow x_3$ ...).
Each computation depends on the previous step, which makes it fundamentally designed for time-series and language data, where order carries meaning.

👉 In NLP: The meaning of a word depends on previous words.

2.Hidden State Memory

RNN maintains a hidden state $h_t$that is updated at every time step using the current input and previous hidden state.

$$h_t = f(x_t, h_{t-1})$$

This hidden state acts as a compressed representation of all past information seen so far.

👉 It serves as the model's "memory", though limited in long-range retention.

3.Parameter Sharing Across Time

The same weight matrices are reused at every time step.
Unlike feedforward networks (where each layer has separate weights), RNN applies one transformation repeatedly across sequence positions.

Benefits:

- Reduces number of parameters
- Allows learning generalized sequential patterns
- Makes model independent of sequence length

ADVANTAGES OF RNN OVER FFN

RNNs model sequential dependencies through a recurrent hidden state, while Feedforward Networks process inputs independently without temporal awareness.

RNNs handle variable-length sequences with shared parameters across time, whereas Feedforward Networks require fixed-size inputs and separate parameters for each position.

RNNs retain contextual information for sequence tasks (e.g., NLP, time-series), while Feedforward Networks lack memory and cannot naturally model order-dependent relationships.

TYPES Of RNN — Based on Input–Output Mapping

Type	Professional Definition	Working Principle	Typical NLP / ML Example
One-to-One	A non-sequential mapping from a single input to a single output (baseline neural setup).	No recurrence across time; behaves like a standard neural network. Included for structural completeness.	Binary classification on fixed feature vectors
One-to-Many	A recurrent setup where a single input generates a sequence of outputs over time.	Initial input initializes hidden state; outputs are generated autoregressively using previous hidden state.	Image Captioning
Many-to-One	A sequence-processing architecture that maps an entire input sequence to a single output.	Hidden state accumulates information across time; final hidden state represents the sequence summary.	Sentiment Analysis
Many-to-Many (Synchronous)	Sequence-to-sequence mapping where input and output sequences are aligned in length.	Output produced at every time step using corresponding hidden state.	POS Tagging, Named Entity Recognition
Many-to-Many (Encoder–Decoder)	Sequence-to-sequence mapping where input and output lengths may differ.	Encoder compresses input sequence into context representation; decoder generates output sequence step-by-step.	Machine Translation

Types of RNN — Based on Architectural Design

1. Vanilla RNN
2. LSTM
3. GRU

DISADVANTAGES OF RNN

1.Vanishing Gradient Problem

During Backpropagation Through Time (BPTT), gradients shrink exponentially as they propagate backward across many time steps.
👉 This prevents the model from learning long-range dependencies.

2.Exploding Gradients

Gradients can grow uncontrollably during training, causing unstable updates and divergence.
👉 Requires techniques like gradient clipping.

3.Poor Long-Term Memory

Although theoretically capable of modelling long dependencies, vanilla RNNs practically retain only short-term information due to gradient decay.

4.Sequential Computation (No Parallelization Across Time)

Each time step depends on the previous hidden state, making computation inherently sequential.
👉 Training is slower compared to Transformer-based models.

METHODS TO PREVENT OVERFITTING DURING TRAINING OF RNN

1.Dropout (Most Important)

Apply dropout:

- Between stacked RNN layers
- On embeddings
- On fully connected output layer

Typical range:

$$p = 0.2 \text{ to } 0.5$$

Why it works:
It prevents co-adaptation of neurons and forces the network to learn more robust representations.

2.Early Stopping (Very Practical)

- Monitor validation loss
- Stop training when it stops improving
- Use patience = 3–5 epochs

Why it works:
Prevents the model from memorizing training data after convergence.

3.Weight Decay (L2 Regularization)

Add in optimizer:

```
optimizer = torch.optim.Adam(model.parameters(), weight_decay=1e-5)
```

Why it works:
Penalizes large weights (especially recurrent weights W_h), reducing model complexity.

4.Reduce Model Capacity

If overfitting persists:

- Reduce hidden size (H)
- Reduce number of layers (L)

Important insight:

$$\text{Parameters} \propto H^2$$

Even small increases in hidden size dramatically increase model capacity.

BIDIRECTIONAL RNN

A Bidirectional Recurrent Neural Network (BiRNN) is an extension of the standard RNN that processes a sequence in both forward (left → right) and backward (right → left) directions, allowing the model to capture information from both past and future context at each time step.